Which model answers the question we are actually asking?

Student notes for lecture 3. One booking, B-001, is described as a user journey, two state models, a data model, a decision table and sequence diagrams, and the models are checked against each other and against the rules. Every scene shows a situation, asks you to predict, reveals the mechanism, changes one condition, compares the outcomes and names the principle.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	What does Ari need to know after pressing Reserve?	Bridge	`week-03.html#/journey-before-boxes`
02	The screen says Unknown. Is the booking unknown too?	Anchor	`week-03.html#/state-machine`
03	When a room changes, which copy of its facts is right?	Anchor	`week-03.html#/identity-and-data`
04	May any signed-in member cancel any booking?	Bridge	`week-03.html#/decision-table`
05	What must happen before the screen may say Confirmed?	Anchor	`week-03.html#/sequence-not-state`
06	The page says “No rooms”. Is the search finished?	Bridge	`week-03.html#/interface-states`
07	A neat diagram sends Confirmed. Where does it contradict R-01?	Bridge	`week-03.html#/model-consistency`
08	Which view answers “Who owns B-001?”	Bridge	`week-03.html#/abstraction-and-zoom`

Campus Rooms is a fictional room-booking service; every name, identifier, time and message is a teaching example unless a source is named. The drawings use a light UML style; they describe the intended behavior at the level of each question, not every detail of an implementation.

One booking, five questions. Which picture answers which?

Week 2 ended with a register of rules. Before drawing anything, look at what people actually need to know about one booking.

The situation

Ari (U-01) pressed Reserve for C101, 10:00 to 11:00. The screen says: “We could not confirm whether your booking was stored. Check My bookings before trying again.”

The whole system, drawn as components

Browser, Booking, Policy, Room Catalog, Notification and Storage. Every box is right, and not one of them tells Ari whether to press Reserve again.

The question of this week

Which model answers the question we are actually asking? (A) One detailed diagram of every part. (B) A model chosen for each question. (C) The code: it is the most precise.

Five questions about B-001

- Ari: Should I press Reserve again? → user journey (scene 1)
- What can happen to B-001 next? → state model (scene 2)
- Room staff: Which room is B-001 for, now that C101 has a new name? → data model (scene 3)
- Bo: May I cancel Ari's booking? → decision table (scene 4)
- Maintainer: What happened before the timeout? → sequence diagram (scene 5)

What this lecture builds

Five small models of one booking, each drawn for the question it answers, and a check that the models agree with each other and with the rules.

What you already know, and where this lecture goes

Which model answers the question we are actually asking?

From Week 2: the register you bring in

R-01 no overlapping confirmations; R-02 duration 30 to 120 minutes; R-03 who may cancel; R-04 notification is separate; R-05 keyboard and status; Q-01 search response time. Week 2 ended on a question: Does a written rule alone make the sequence of booking, cancellation and failure states understandable?

The example this week

Ari U-01 and Bo U-02; rooms C101 and C202; B-001: Ari's booking of C101, 10:00 to 11:00, Confirmed. A stored booking is Confirmed or Cancelled. Draft, Submitting, Rejected and Unknown outcome belong to the request on the screen, and a rejected request never creates a confirmed booking.

How the drawings work

Light UML-style drawings, introduced where they are first used: rounded boxes are states, an arrow carries an event and its condition, a lifeline carries numbered messages.

By the end of this lecture you can

1. Choose a model suited to a behavior, structure, or interaction question.
2. Read a state transition with an event, condition, and outcome.
3. Distinguish a stored booking from a transient interface or request state.
4. Explain entity identity and relationships without duplicating shared facts.
5. Locate a contradiction between a requirement, a diagram, and an interface message.

What does Ari need to know after pressing Reserve?

Panel 1 of 2 · One journey: the message and the stored state on separate rows, and the three outcomes

Condition: **Baseline: the screen tells Ari each outcome** State: **6 of 7**

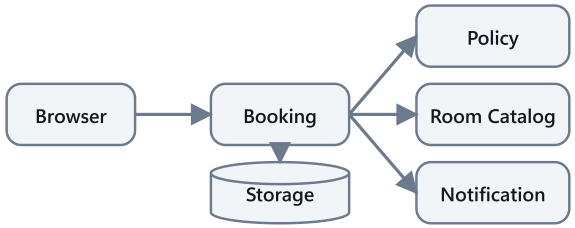
The same booking as components: correct boxes, correct arrows, and no place for what Ari must be told next. The journey found that need first.

	1 Choose a room	2 Submit the request	3 Observe the outcome
GOAL	find a free room from 10:00 to 11:00	ask for the room	know whether the room is held
ACTION	selects C101, 10:00 to 11:00	presses Reserve	reads the result
THE SCREEN SAYS	C101 is free from 10:00 to 11:00	Sending your request...	Booking confirmed: B-001, C101, 10:00 to 11:00
STORED UNDERNEATH	nothing stored yet	nothing decided yet	B-001 Confirmed
STILL UNCERTAIN	someone else can still book it first	sent is not the same as confirmed	nothing about the room; the email may still be on its way (R-04)

THE OUTCOME STEP CAN END THREE WAYS

OUTCOME	THE SCREEN SAYS	STORED	NEXT FOR ARI
booking confirmed	Booking confirmed: B-001, C101, 10:00 to 11:00	B-001 Confirmed	nothing to do; the room is held
not available	Not available: C101 is already booked from 10:00 to 11:00	nothing stored for Ari	choose another time or room
could not determine the result	We could not confirm whether your booking was stored. Check My bookings before trying again.	B-001 may be Confirmed; the screen cannot tell	check My bookings before pressing Reserve again

THE SAME BOOKING AS COMPONENTS



No box says what Ari must be told next.

Choose a room, submit the request, observe the outcome. “Sending your request...” means the request was sent, not that the booking is confirmed. The outcome step ends as booking confirmed (next: nothing to do; the room is held); not available (next: choose another time or room); could not determine the result (next: check My bookings before pressing Reserve again). A component view of Browser, Booking, Policy, Room Catalog, Notification and Storage is correct and still has no place for what Ari must be told.

What does Ari need to know after pressing Reserve?

Panel 2 of 2 · The same journey when the confirmation message is missing

Condition: **Changed: B-001 is stored, no message is shown** State: **4 of 5**

The duplicate click misleads: Ari concludes that someone else holds the room, the room Ari already holds.

	1 Choose a room	2 Submit the request	3 Observe the outcome
GOAL	find a free room from 10:00 to 11:00	ask for the room	know whether the room is held
ACTION	selects C101, 10:00 to 11:00	presses Reserve	reads the result
THE SCREEN SAYS	C101 is free from 10:00 to 11:00	Sending your request...	*the form returns to its start, with no message*
STORED UNDERNEATH	nothing stored yet	nothing decided yet	B-001 Confirmed
STILL UNCERTAIN	someone else can still book it first	sent is not the same as confirmed	Ari cannot tell: maybe it did not work

ARI PRESSES RESERVE AGAIN

Request: C101, 10:00 to 11:00, the same request again

R-01: rejected: it overlaps B-001

Not available: C101 is already booked from 10:00 to 11:00

The consequence: Ari concludes that someone else holds the room, the room Ari already holds.

TWO PICTURES OF THE SAME MOMENT

The system: B-001 Confirmed

Ari: "Maybe it did not work?"

B-001 is stored, but no confirmation message is shown. The system knows B-001 Confirmed; Ari thinks "maybe it did not work". Pressing Reserve again sends the same request, which R-01 rejects because it overlaps B-001: "Not available: C101 is already booked from 10:00 to 11:00" Ari concludes that someone else holds the room, the room Ari already holds.

Principle · A user journey exposes information needs that an internal component diagram can miss.

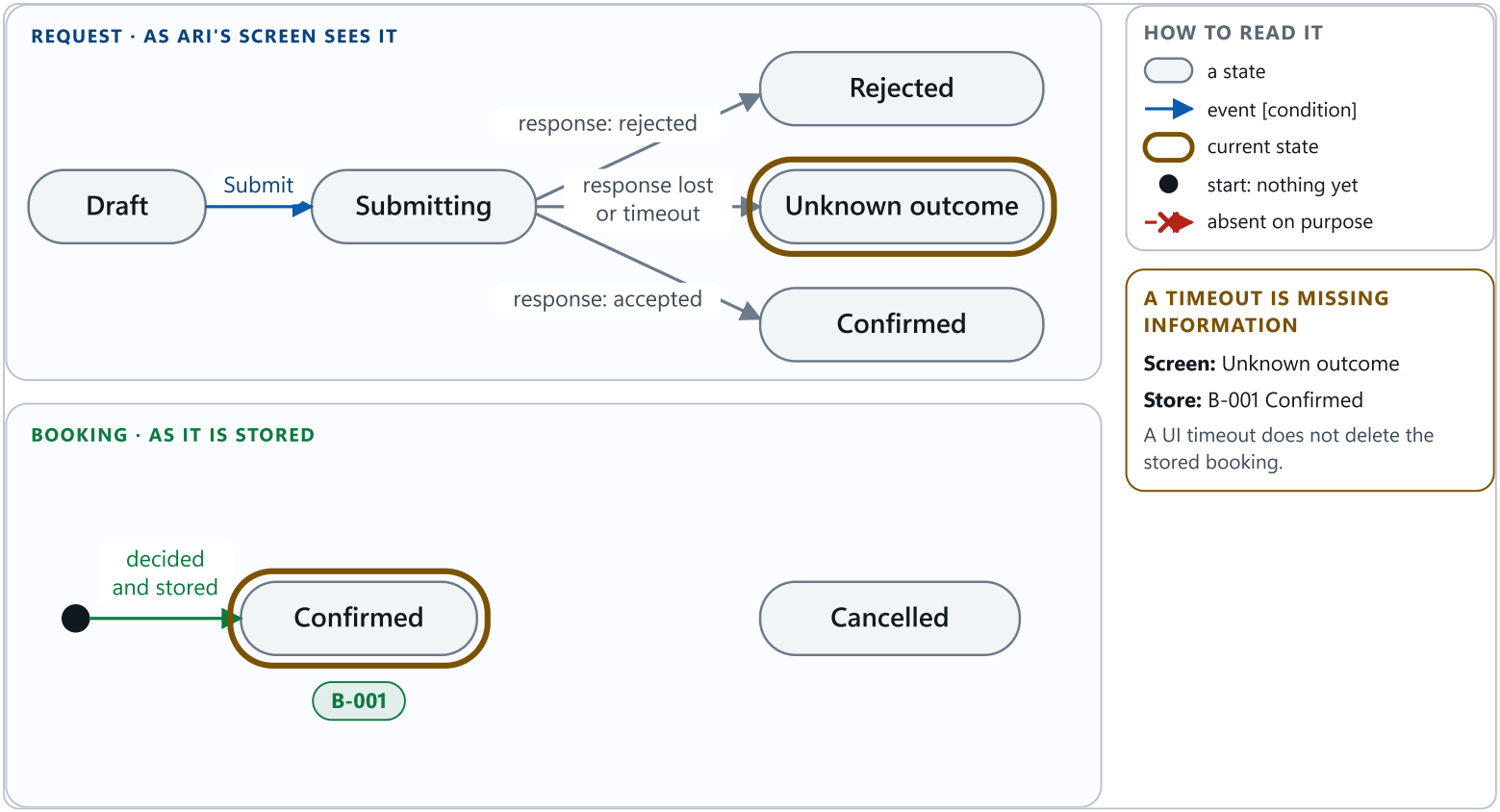
In short · Follow the person first; boxes cannot show what they need.

The screen says Unknown. Is the booking unknown too?

Panel 1 of 3 · Two state models: the three ways out of Submitting, and a lost response

Condition: **Baseline: one request from Draft to its outcome** State: **6 of 8**

Now the response is lost. The screen can only show **Unknown outcome**, while the stored booking is still **Confirmed**. A timeout is missing information.



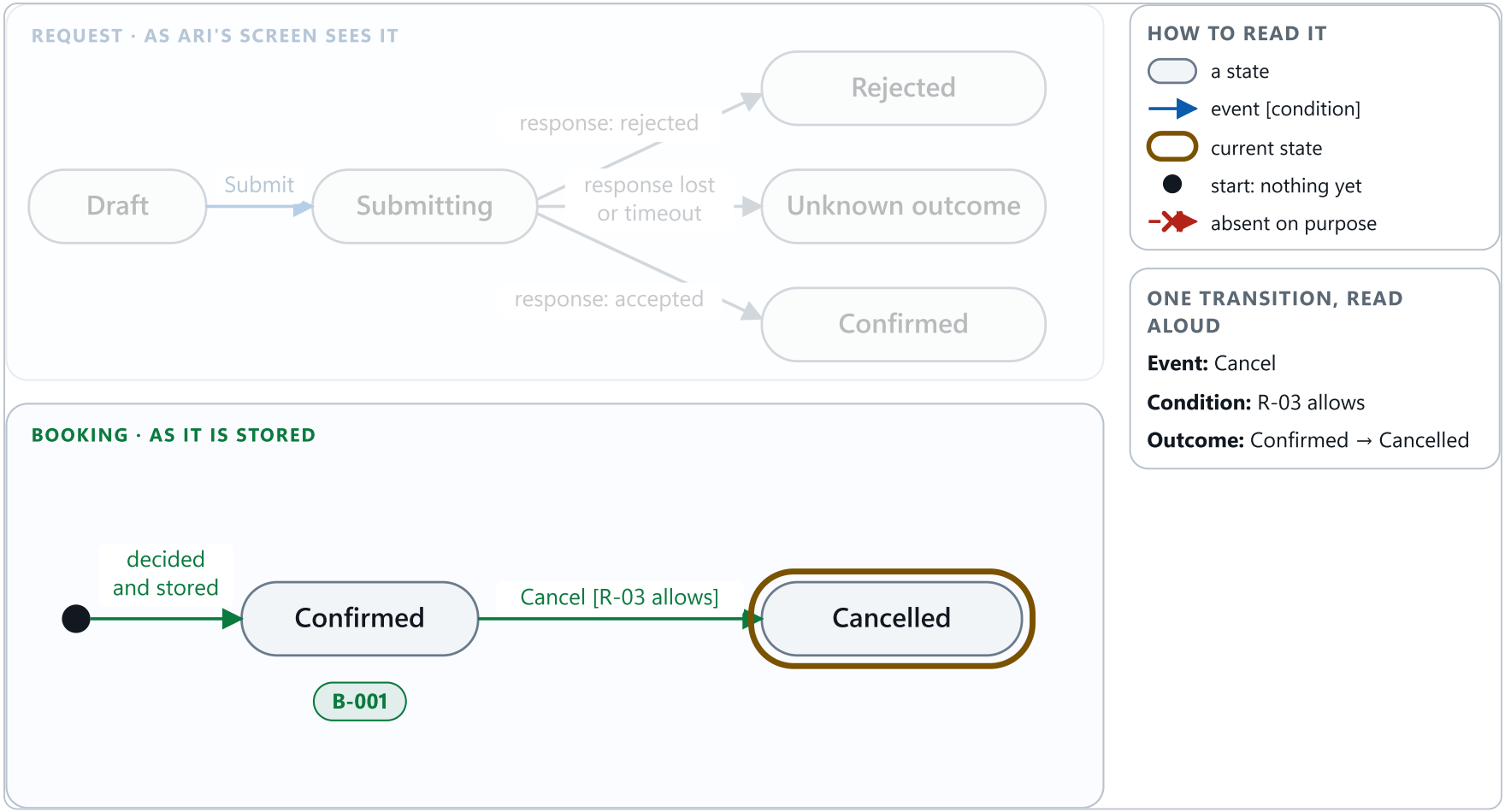
The request model: Draft → Submitting on “Submit”; Submitting → Confirmed on “response: accepted”; Submitting → Rejected on “response: rejected”; Submitting → Unknown outcome on “response lost or timeout”. The stored model: an accepted request creates B-001 Confirmed; a rejected request stores nothing. When the response is lost, the screen shows Unknown outcome while the stored booking is still Confirmed. A timeout is missing information: the booking can exist while the screen does not know it, and a UI timeout does not delete the stored booking.

The screen says Unknown. Is the booking unknown too?

Panel 2 of 3 · One stored transition: event, condition, outcome

Condition: **Baseline: one request from Draft to its outcome** State: **7 of 8**

Read one transition: the **event** is Cancel, the **condition** is R-03 allows, the **outcome** is Cancelled. Only the stored model changes.



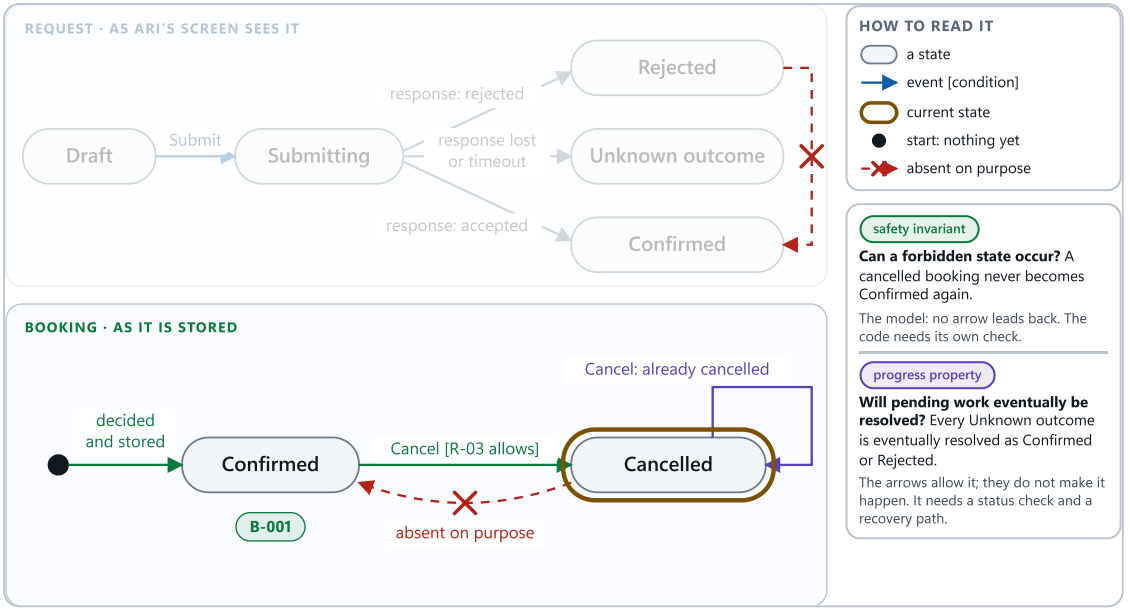
Event Cancel, condition R-03 allows, outcome Confirmed → Cancelled. Only the stored model changes; the request model belongs to the screen.

The screen says Unknown. Is the booking unknown too?

Panel 3 of 3 · Cancel again, the arrows left out on purpose, and two kinds of question

Condition: **Changed: B-001 is Cancelled; Cancel arrives again** State: **4 of 5**

Two kinds of question: **can a forbidden state occur?** is about safety, **will pending work eventually be resolved?** is about progress. The arrows answer the first for the model only.



A second Cancel for a cancelled booking gets the defined answer “Already cancelled. Nothing changed.”, with no transition. The accident would be a second state change: the history shows two cancellations and the owner receives a second notice. Absent on purpose: Cancelled → Confirmed, Rejected → Confirmed. Safety invariant: Can a forbidden state occur? (A cancelled booking never becomes Confirmed again). Progress property: Will pending work eventually be resolved? (Every Unknown outcome is eventually resolved as Confirmed or Rejected). Arrows show what is allowed; whether the code keeps to them, and whether pending work finishes, needs checks and a recovery path of their own.

Principle · State models clarify which changes are possible, what triggers them, and which facts are actually known.

In short · Show what can change, what triggers it, and what is known.

When a room changes, which copy of its facts is right?

Panel 1 of 3 · Copied capacity: two records disagree with the room

Condition: **Baseline: each booking copies the room's facts** State: **2 of 6**

The room changes to **10**. Both copies still say **12**: two records now disagree with the room, and neither knows which value is right.

ROOM C101

id	C101
display_name	Seminar Room 1
capacity	10

BOOKING B-001 Confirmed

owner_id	U-01 (Ari)
room_id	C101
time	10:00 to 11:00
capacity (copy)	12 ✕ **disagrees with the room**

BOOKING B-003 Confirmed

owner_id	U-02 (Bo)
room_id	C101
time	12:00 to 13:00
capacity (copy)	12 ✕ **disagrees with the room**

B-001 and B-003 each copied the capacity of C101 (12). When the room changes to 10, both copies still say 12: the copies disagree with the room.

When a room changes, which copy of its facts is right?

Panel 2 of 3 · One identity per room: entities, references and many-to-one relationships

Condition: **Baseline: each booking copies the room's facts** State: **5 of 6**

Read the lines: a booking belongs to **one** user and **one** room; a user or a room can have **many** bookings over time. C101 already has two.

ROOM C101

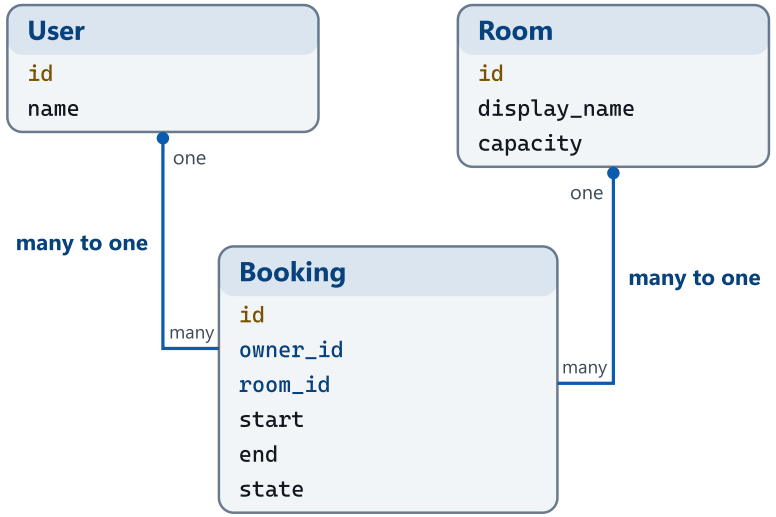
id	C101
display_name	Seminar Room 1
capacity	10

BOOKING B-001 Confirmed

owner_id	U-01 (Ari)
room_id	C101
time	10:00 to 11:00
capacity	read from C101: 10

BOOKING B-003 Confirmed

owner_id	U-02 (Bo)
room_id	C101
time	12:00 to 13:00
capacity	read from C101: 10



THE INSTANCES

U-01 Ari · U-02 Bo · C101 Seminar Room 1

`B-001 → owner U-01, room C101`

`B-003 → owner U-02, room C101`

One room, two bookings; each booking has one owner and one room.

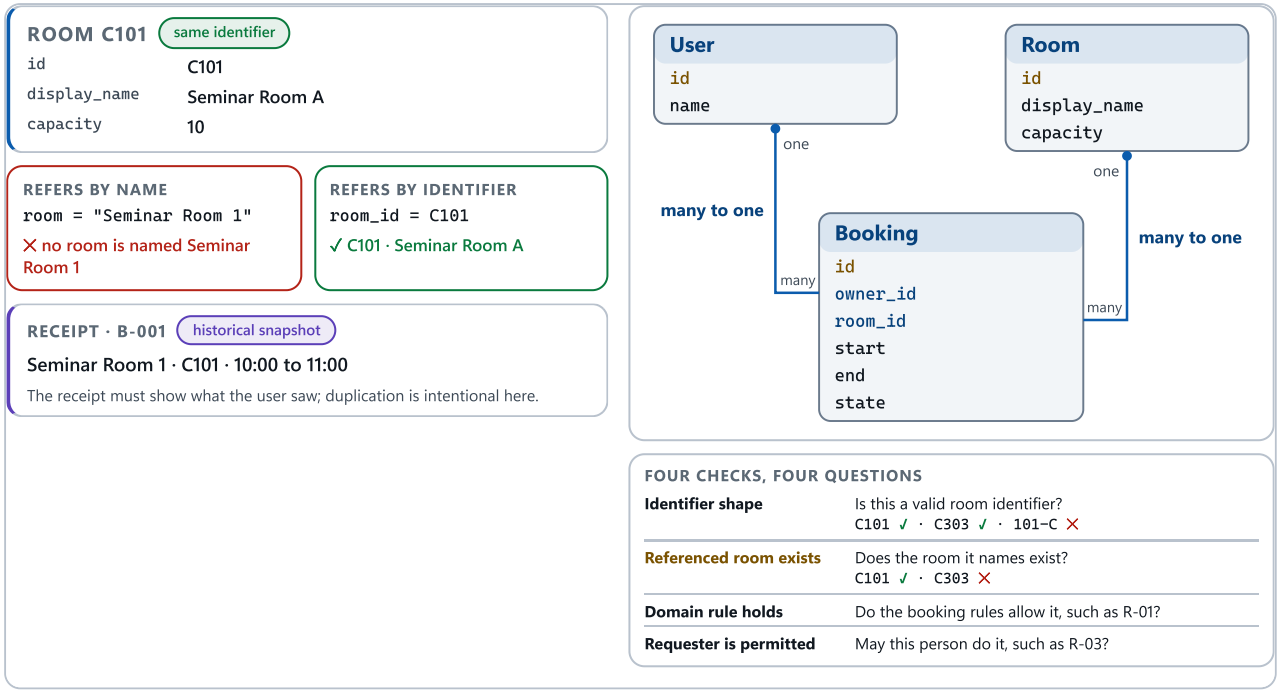
User (id, name), Room (id, display_name, capacity), Booking (id, owner_id, room_id, start, end, state). A booking refers to its owner and room by identifier: Booking to User, many to one; Booking to Room, many to one. The capacity is read from the room, so there is one current value. An entity in this model does not have to become exactly one class or one table.

When a room changes, which copy of its facts is right?

Panel 3 of 3 · A new name, the same identifier, a deliberate snapshot and four different checks

Condition: **Changed: C101 gets a new name; its ID stays** State: **4 of 5**

Well-formed is not **exists**: C303 has the right shape and names no room. References that always point at a real room are **referential integrity**.



C101 is renamed from Seminar Room 1 to Seminar Room A. A reference by name finds no room is named Seminar Room 1; a reference by C101 still finds the room. The receipt of B-001 keeps “Seminar Room 1” because the receipt must show what the user saw; duplication is intentional here. Identifier shape (one capital letter followed by three digits, such as C101) and existence are different checks: C101 well-formed, exists; C303 well-formed, no such room; 101-C malformed, no such room. The booking rules (R-01) and the permission (R-03) are two more separate questions.

Principle · Model identity and the meaning of a fact before deciding where to store it.

In short · Decide what a fact means before deciding where it lives.

May any signed-in member cancel any booking?

Panel 1 of 2 · R-03 as rows: every core combination, and the gap

Condition: **Baseline: R-03 written as rows** State: **8 of 8**

R-03 allows exactly one combination; every other row says why not.

Rule R-03 **An authenticated requester may cancel their own future confirmed booking.**

#	WHO ASKS	SIGNED IN	OWN BOOKING	BOOKING STATE	STARTS AFTER NOW	DECISION	WHY
1	Ari · U-01	✓	✓	Confirmed	✓ now 09:00, start 10:00	allowed	Every condition holds
2	Bo · U-02	✓	✗	Confirmed	✓ now 09:00, start 10:00	denied	Not the owner
3	Ari · U-01	✓	✓	Confirmed	✗ now 10:00, start 10:00	denied	Start is not later than now
4	nobody signed in	✗	·	Confirmed	✓ now 09:00, start 10:00	denied	No authenticated requester
5	Ari · U-01	✓	✓	Cancelled	✓ now 09:00, start 10:00	repeated response	Stable, defined answer; no second transition
?	Ari · U-01	✓	✓	Confirmed	✗ now 12:00, start 10:00	denied	Missing row: start is not later than now: the booking has already ended

Policy boundary: R-03 allows one combination: signed in, own booking, Confirmed, starting after now. Every other row is denied or answered with the repeated response; a staff override would be a new rule decided by the stakeholders.

R-03: An authenticated requester may cancel their own future confirmed booking. Conditions: Signed in, Own booking, Booking state, Starts after now. Rows: Ari · U-01, Confirmed, now 09:00: allowed; Bo · U-02, Confirmed, now 09:00: denied; Ari · U-01, Confirmed, now 10:00: denied; nobody signed in, Confirmed, now 09:00: denied; Ari · U-01, Cancelled, now 09:00: repeated response; Ari · U-01, Confirmed, now 12:00: denied. Policy boundary: R-03 allows one combination: signed in, own booking, Confirmed, starting after now. Every other row is denied or answered with the repeated response; a staff override would be a new rule decided by the stakeholders.

Principle · A decision table makes combinations and missing cases easier to inspect.

In short · Rows show every combination, and the gaps between them.

May any signed-in member cancel any booking?

Panel 2 of 2 · Only the requester changes

Condition: **Changed: only who asks changes; all else fixed** State: **4 of 5**

Room staff, signed in: **denied** under R-03. A staff override would be a new rule for the stakeholders to decide, not an exception hidden in code.

Rule R-03 **An authenticated requester may cancel their own future confirmed booking.**

Held fixed: signed in ✓ · Confirmed · now 09:00, start 10:00 ✓. Only the second column changes.

#	WHO ASKS	SIGNED IN	OWN BOOKING	BOOKING STATE	STARTS AFTER NOW	DECISION	WHY
1	Ari · U-01	✓	✓	Confirmed	✓ now 09:00, start 10:00	allowed	Every condition holds
2	Bo · U-02	✓	✗	Confirmed	✓ now 09:00, start 10:00	denied	Not the owner
3	Room staff	✓	✗	Confirmed	✓ now 09:00, start 10:00	denied	Not the owner; R-03 has no staff override

No staff override: Not the owner; R-03 has no staff override. Campus staff have no unstated right to cancel every booking.

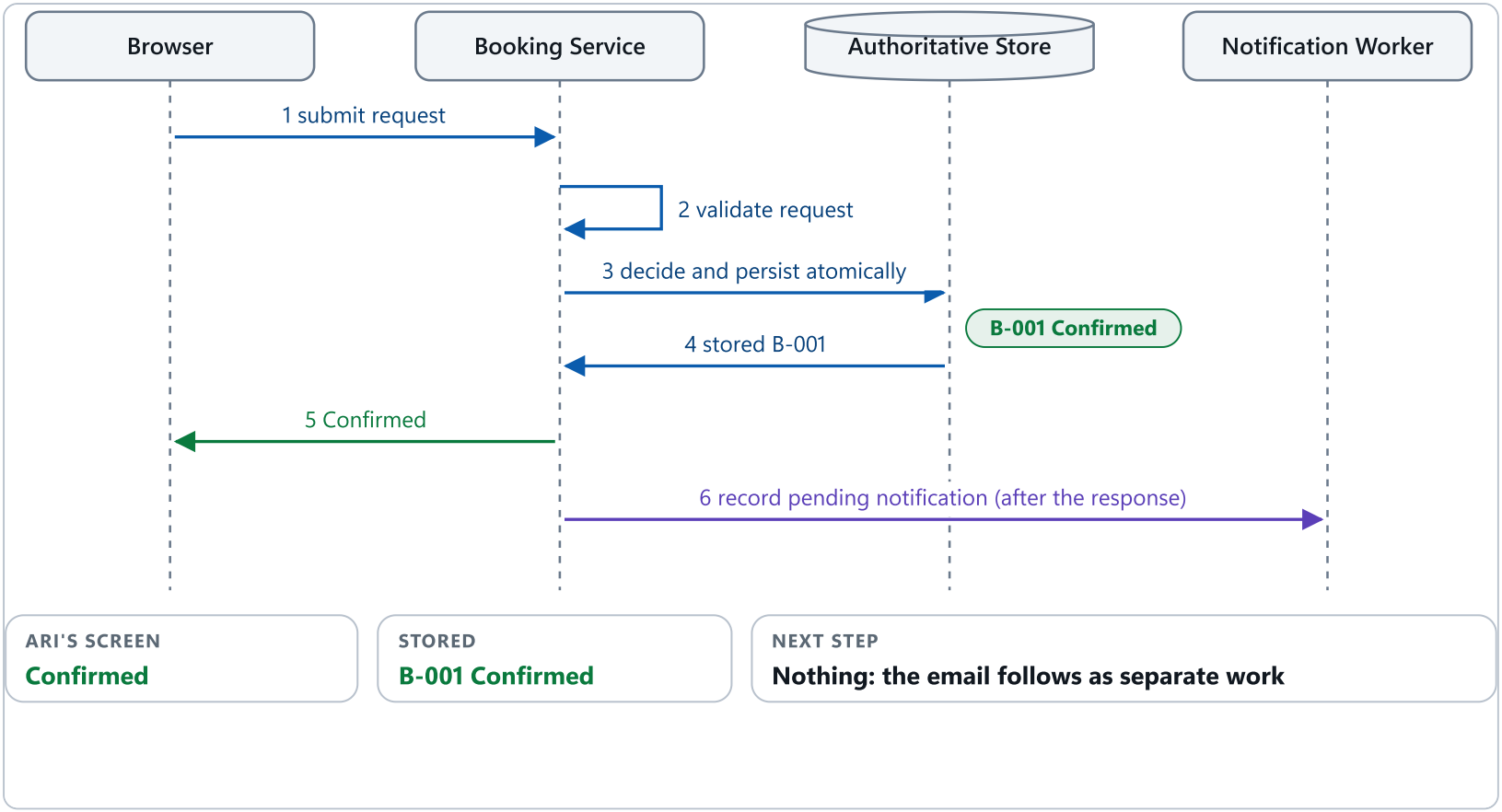
Signed in, Confirmed, now 09:00 before start 10:00: Ari · U-01 allowed, Bo · U-02 denied, Room staff denied. The phrase “their own” in R-03 decides it; there is no staff override.

What must happen before the screen may say Confirmed?

Panel 1 of 3 · The successful sequence: decide and store before Confirmed; notification after the response

Condition: **Baseline: one booking succeeds, message by message** State: **7 of 8**

6 record pending notification, after the response: separate work with its own state (R-04). The confirmation never waits for the email.



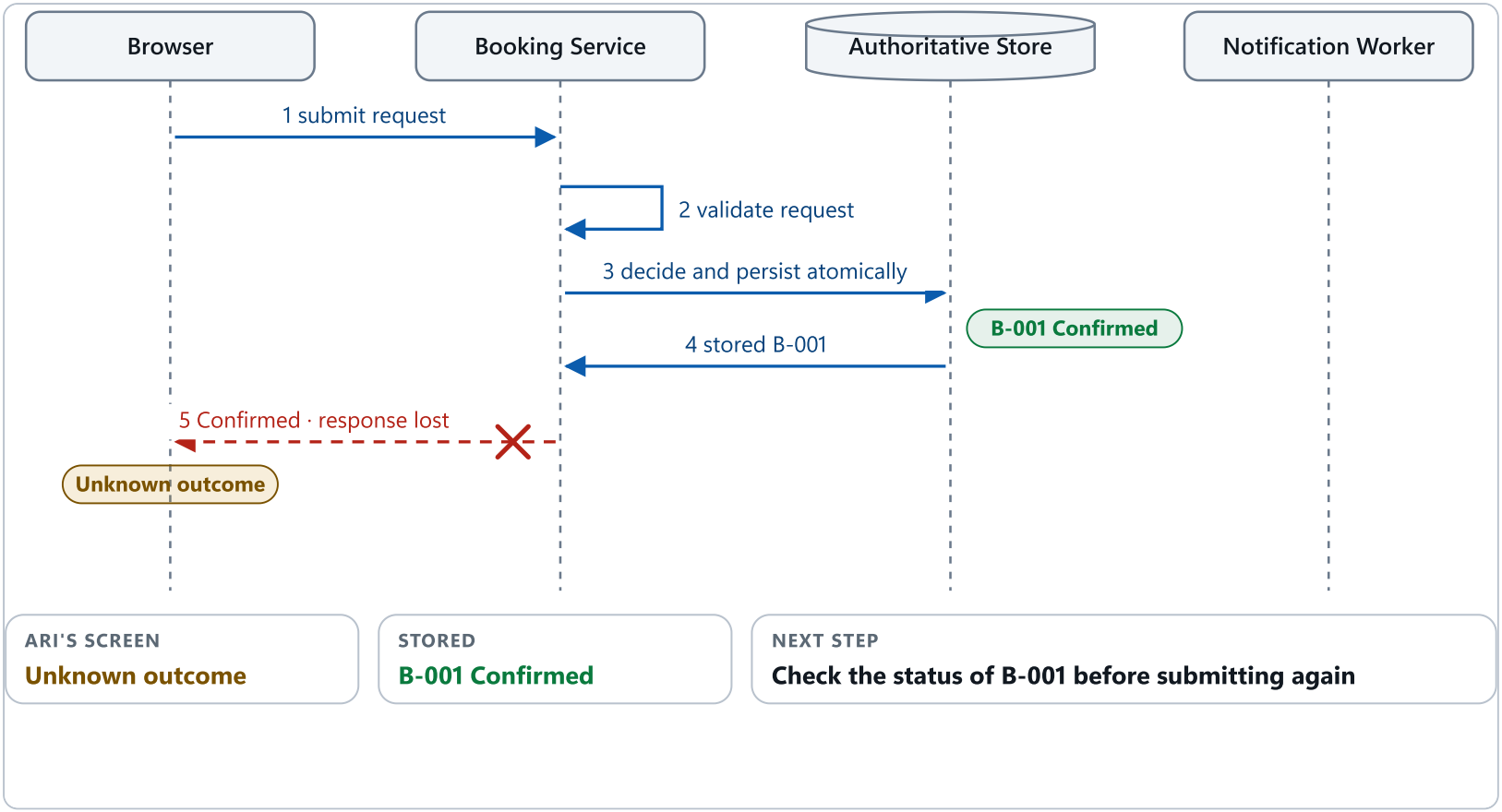
1 Browser → Booking Service: submit request; 2 Booking Service → Booking Service: validate request; 3 Booking Service → Authoritative Store: decide and persist atomically; 4 Authoritative Store → Booking Service: stored B-001; 5 Booking Service → Browser: Confirmed; 6 Booking Service → Notification Worker: record pending notification (after the response). The numbers give the order; vertical distance is not elapsed time.

What must happen before the screen may say Confirmed?

Panel 2 of 3 · The lost-response sequence: stored Confirmed, screen Unknown outcome

Condition: **Changed: B-001 is stored, the response is lost** State: **4 of 5**

The sequence does: message 5 is where they differ. So the recovery is to **check the status of B-001 before submitting again**, not to send a second booking.



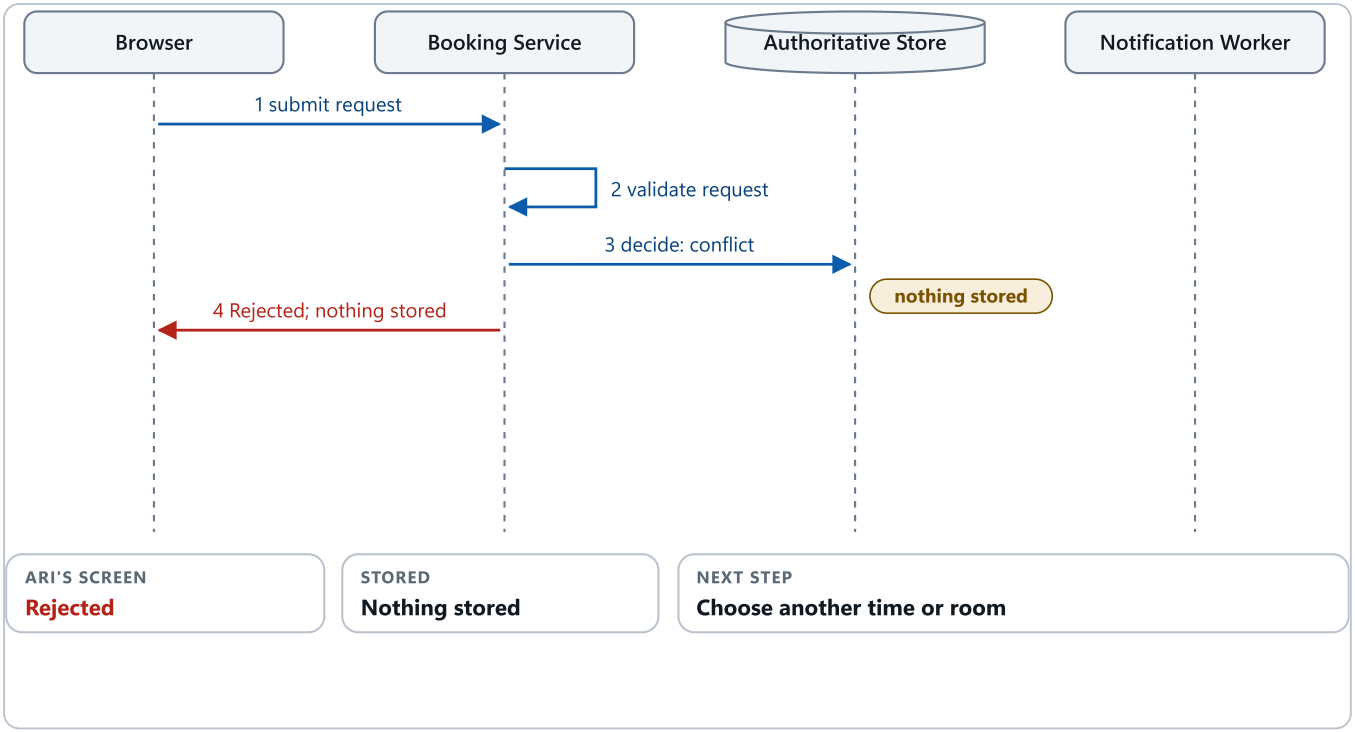
Messages 1 to 4 as in the success; message 5 is lost. The browser shows Unknown outcome while the stored booking is Confirmed. Recovery: check the status of B-001 before submitting again.

What must happen before the screen may say Confirmed?

Panel 3 of 3 · Rejected before persistence: nothing stored, a different recovery

Condition: **Changed: rejected before anything is stored** State: **4 of 4**

Two people without a confirmation, two histories, two different next steps.



1 Browser → Booking Service: submit request; 2 Booking Service → Booking Service: validate request; 3 Booking Service → Authoritative Store: decide: conflict; 4 Booking Service → Browser: Rejected; nothing stored. Recovery: choose another time or room; nothing was stored, so there is nothing to check. Both people see no confirmation; the histories, and so the next steps, differ.

Principle · An interaction history explains why two apparently similar user symptoms can require different recovery actions.

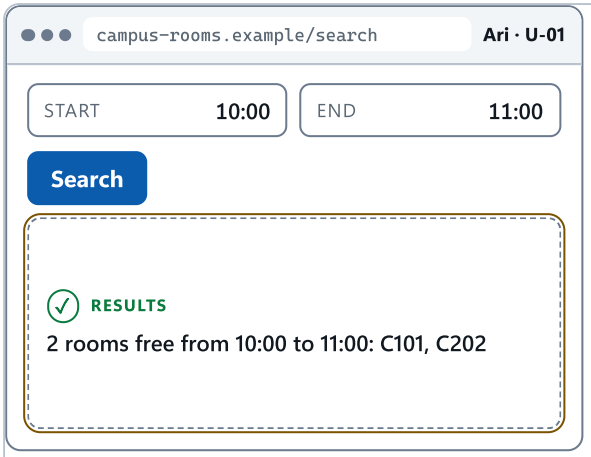
In short · Same symptom, different history, different fix.

The page says “No rooms”. Is the search finished?

Panel 1 of 2 · One search screen in five states: what is known and what to do next

Condition: **Baseline: five states, five different messages** State: **6 of 7**

Results: the search finished; these rooms were free when it ran: C101 and C202. Next: choose a room.



ONE SCREEN, FIVE PROMISES

STATE	WHAT IS KNOWN	NEXT ACTION	KEYBOARD FOCUS
Loading	The question is being answered; nothing is known yet	Wait, or cancel the search	Stays on the search form
Empty result	The search finished; no room is free at that time	Change the time	Moves to the message
Validation error	Nothing was searched; one field needs fixing	Correct the end time	Moves to the end-time field
Service unavailable	The search did not complete	Try again	Moves to the message and its Try again button
Results	The search finished; these rooms were free when it ran	Choose a room	Moves to the results heading

Loading: “Searching for rooms free from 10:00 to 11:00…” (the question is being answered; nothing is known yet; next: wait, or cancel the search; focus stays on the search form) · Empty result: “No room is free from 10:00 to 11:00. Try another time.” (the search finished; no room is free at that time; next: change the time; focus moves to the message) · Validation error: “End time must be later than start time.” (nothing was searched; one field needs fixing; next: correct the end time; focus moves to the end-time field) · Service unavailable: “We could not load rooms. Nothing is known about free rooms yet.” (the search did not complete; next: try again; focus moves to the message and its Try again button) · Results: “2 rooms free from 10:00 to 11:00: C101, C202” (the search finished; these rooms were free when it ran; next: choose a room; focus moves to the results heading)

The page says “No rooms”. Is the search finished?

Panel 2 of 2 · Empty result versus failed request, with and without color

Condition: **Changed: empty result versus failed request** State: **4 of 5**

Take the color away: the two states still differ by their **words** and their **icon shape**. That is what rule R-05 needs: status you can perceive.



COMPARE	EMPTY RESULT	SERVICE UNAVAILABLE
What is known	The search finished; no room is free at that time	The search did not complete
Next action	Change the time	Try again
Keyboard focus	Moves to the message	Moves to the message and its Try again button
Without color	The words and the circle icon	The words and the cross icon

Empty result: “No room is free from 10:00 to 11:00. Try another time.” Service unavailable: “We could not load rooms. Nothing is known about free rooms yet.” The next actions differ (change the time; try again); the states differ by words and icon shape, not by color alone.

Principle · An interface should communicate what is known and what the user can do next.

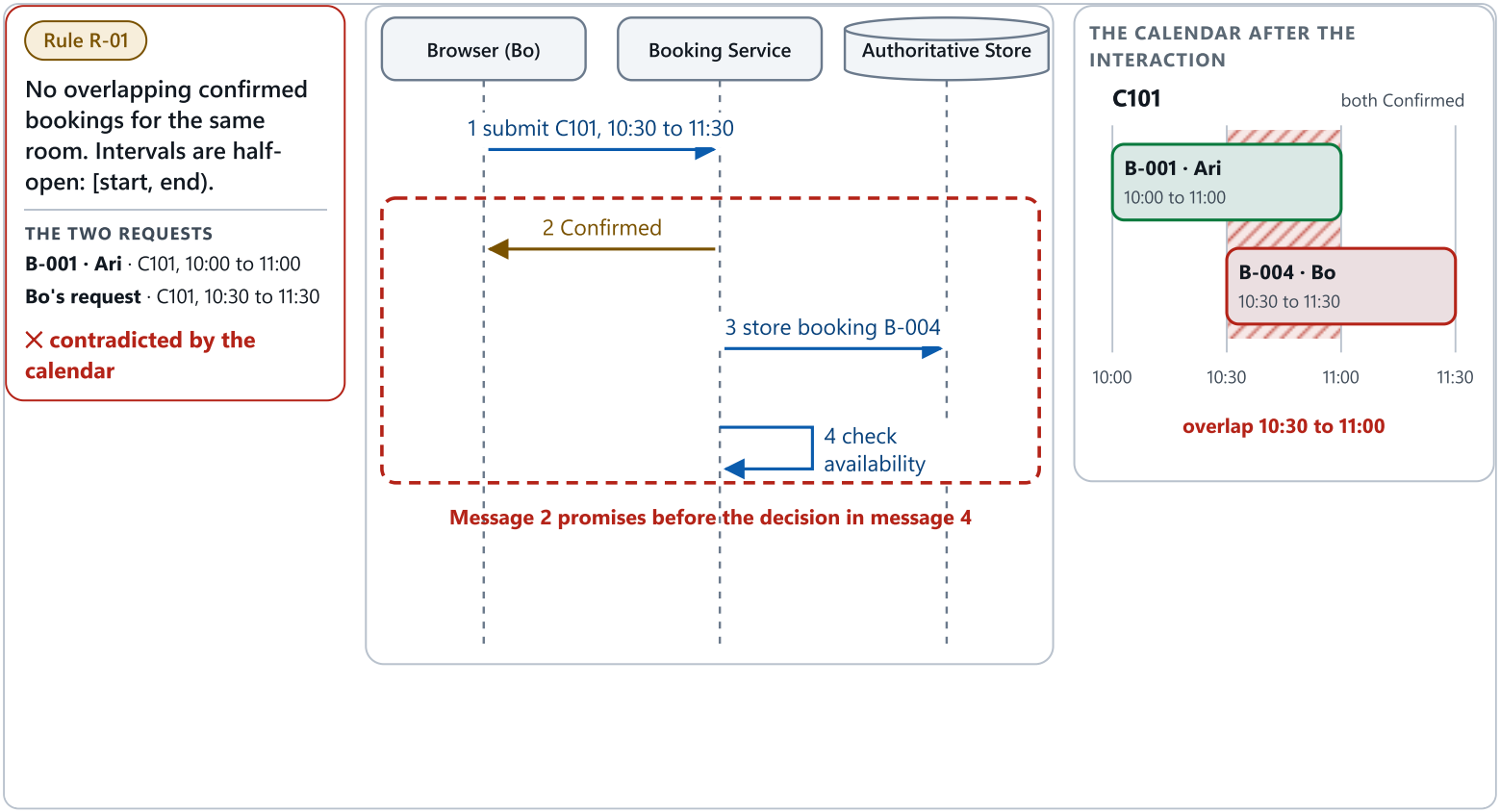
In short · Say what is known and what the person can do.

A neat diagram sends Confirmed. Where does it contradict R-01?

Panel 1 of 2 · The flawed sequence: Confirmed before the decision, and the overlapping calendar

Condition: **Baseline: the diagram confirms before deciding** State: **5 of 6**

The contradiction: R-01 forbids that overlap, and the message order allowed it. **Message 2 comes before the decision.** Neat, and wrong.



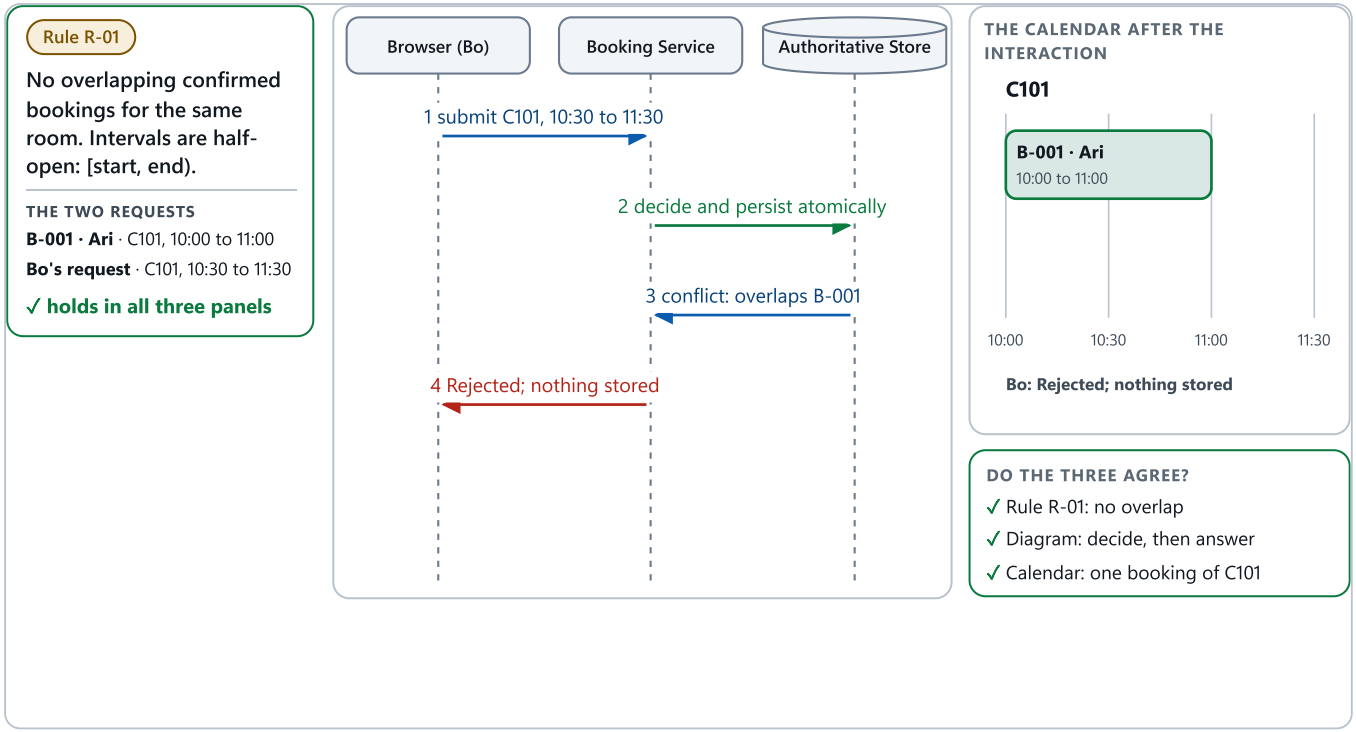
R-01: No overlapping confirmed bookings for the same room. Intervals are half-open: [start, end). 1 Browser (Bo) → Booking Service: submit C101, 10:30 to 11:30; 2 Booking Service → Browser (Bo): Confirmed; 3 Booking Service → Authoritative Store: store booking B-004; 4 Booking Service → Booking Service: check availability. Result: two confirmed bookings of C101 overlap from 10:30 to 11:00. The contradiction: message 2 promises Confirmed before any availability decision; the check in message 4 comes after the promise and after the store.

A neat diagram sends Confirmed. Where does it contradict R-01?

Panel 2 of 2 · The corrected sequence: decide, then answer; the three panels agree

Condition: **Changed: Confirmed moves after the decision** State: **3 of 4**

3 conflict: overlaps B-001, so **4 Rejected; nothing stored**. The calendar keeps one booking; the rule, the diagram and the calendar agree.



1 Browser (Bo) → Booking Service: submit C101, 10:30 to 11:30; 2 Booking Service → Authoritative Store: decide and persist atomically; 3 Authoritative Store → Booking Service: conflict: overlaps B-001; 4 Booking Service → Browser (Bo): Rejected; nothing stored. The calendar keeps only B-001; rule, diagram and calendar agree. Neatness is equal in both diagrams; consistency is not.

Principle · Models are useful when their claims can be checked against one another and against intended behavior.

In short · Walk one example through every model; they must agree.

Which view answers “Who owns B-001?”

Panel 1 of 2 · Four views of one booking: the question-to-model matrix

Condition: **Baseline: a student asks about the booking** State: **7 of 7**

One question per view, and each view says what it leaves out.

VIEW [Context] [Sequence] [State] [Data]

ONE QUESTION PER VIEW

VIEW	ANSWERS	KEEPS	LEAVES OUT
Context	Who uses Campus Rooms, and what lies outside it?	People, the service boundary, outside systems	States, messages, fields
Sequence	What happened before the timeout?	Participants and the numbered order of messages	Fields, ownership, other histories
State	What can happen to B-001 next?	States, events and conditions	Who asked, message order, timing
Data	Who owns B-001?	Identity, references, cardinality	Order of events, screen messages

Context: answers “Who uses Campus Rooms, and what lies outside it?”; keeps people, the service boundary, outside systems; leaves out states, messages, fields. Sequence: answers “What happened before the timeout?”; keeps participants and the numbered order of messages; leaves out fields, ownership, other histories. State: answers “What can happen to B-001 next?”; keeps states, events and conditions; leaves out who asked, message order, timing. Data: answers “Who owns B-001?”; keeps identity, references, cardinality; leaves out order of events, screen messages. “Who owns B-001?” → owner_id U-01, which is Ari. “What happened before the timeout?” → B-001 was stored at message 4; the response, message 5, was lost.

Principle · A useful abstraction preserves what matters for its question and makes its omissions intentional.

In short · Keep what the question needs; leave the rest out on purpose.

Which view answers “Who owns B-001?”

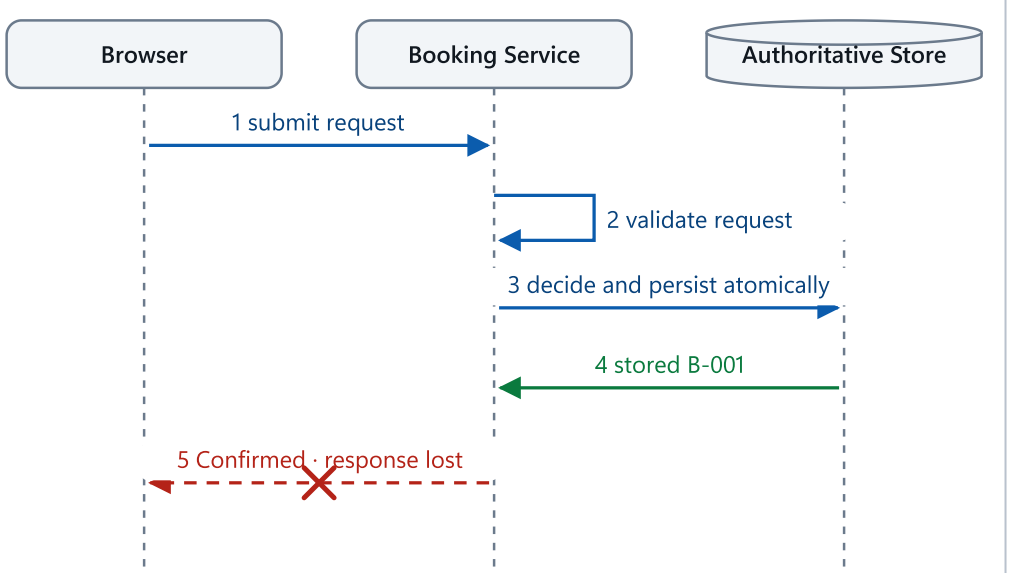
Panel 2 of 2 · A different audience: what the maintainer and the student each need

Condition: **Changed: a maintainer investigates the failure** State: **3 of 4**

The student needs **the message on the screen and the next action**, and can leave out participants, message numbers, field names. Same booking, different useful detail.

VIEW Context | Sequence | State | Data

What happened before the timeout?



Answer: B-001 was stored at message 4; the response, message 5, was lost.

A MAINTAINER INVESTIGATING THE FAILURE

Needs: the numbered messages and the stored state of B-001

Leaves out: the room's display name and the wording on the screen

A STUDENT USING THE SERVICE

Needs: the message on the screen and the next action

Leaves out: participants, message numbers, field names

A maintainer investigating the failure needs the numbered messages and the stored state of B-001 and can leave out the room's display name and the wording on the screen. A student using the service needs the message on the screen and the next action and can leave out participants, message numbers, field names.

Five models of one booking, and the question each answers

The notation of this lecture in one place: what each model shows, the question it answers and what it leaves out, for one and the same booking.

MODEL	SCENE	SHOWS	ANSWERS	LEAVES OUT
User journey	1	steps, goals, what the screen says, what is stored and what is still uncertain	What does the person need to know next?	components, messages and field names
State models	2	states, events and conditions, for the request and for the stored booking	What can happen next, and on which event?	who asked, the order of messages, timing
Data model	3	entities, identifiers, references and how many of each	Which facts exist, and where does each one live?	the order of events and the screen messages
Decision table	4	one row per combination of conditions, with its decision	Which combination is allowed, and why?	time order and participants
Sequence diagram	5	participants and numbered messages, top to bottom	In what order did it happen, and where did it stop?	fields and the other possible histories

Notation used this week

- a state: rounded box
- event [condition] on an arrow
- the current state
- start: nothing stored yet
- absent on purpose
- entity: fields, id first
- many to one
- a lost message

Eight principles, six checks

One principle per scene, then six checks: decide your answer before you open each one.

Principles

1 · BRIDGE

A user journey exposes information needs that an internal component diagram can miss.

In short: Follow the person first; boxes cannot show what they need.

`week-03.html#/journey-before-boxes`

2 · ANCHOR

State models clarify which changes are possible, what triggers them, and which facts are actually known.

In short: Show what can change, what triggers it, and what is known.

`week-03.html#/state-machine`

3 · ANCHOR

Model identity and the meaning of a fact before deciding where to store it.

In short: Decide what a fact means before deciding where it lives.

`week-03.html#/identity-and-data`

4 · BRIDGE

A decision table makes combinations and missing cases easier to inspect.

In short: Rows show every combination, and the gaps between them.

`week-03.html#/decision-table`

5 · ANCHOR

An interaction history explains why two apparently similar user symptoms can require different recovery actions.

In short: Same symptom, different history, different fix.

`week-03.html#/sequence-not-state`

6 · BRIDGE

An interface should communicate what is known and what the user can do next.

In short: Say what is known and what the person can do.

`week-03.html#/interface-states`

7 · BRIDGE

Models are useful when their claims can be checked against one another and against intended behavior.

In short: Walk one example through every model; they must agree.

`week-03.html#/model-consistency`

8 · BRIDGE

A useful abstraction preserves what matters for its question and makes its omissions intentional.

In short: Keep what the question needs; leave the rest out on purpose.

`week-03.html#/abstraction-and-zoom`

Checks: decide your answer before reading the one below each question

1. A response times out after persistence. Is the booking necessarily absent?

> No. The requester may not know the result even though the booking exists.

2. Why reference a room ID instead of copying its current name everywhere?

> Identity remains stable when the displayed name changes; historical snapshots should be modeled deliberately.

3. Which model best exposes combinations of ownership and status?

> A decision table.

4. How can a diagram be tested before code exists?

> Walk concrete examples through it and compare outcomes with requirements and complementary models.

5. Can a valid state diagram establish that pending notification work will eventually complete?

> No. Permitted states and transitions do not establish the scheduling and recovery assumptions needed for progress.

6. A mobile note says “saved on this device” while synchronization is pending. Does that contradict the cloud copy being older?

> No. Local persistence and remote synchronization are separate states. Model both and make the message identify which promise has been met; pending synchronization still needs a defined completion or recovery path.

What carries forward, and where to read more

Next week asks how a team organizes the work of turning these models into dependable behavior, and learns where it waits.

Week 4 starts from this question:

Who turns these models into working behavior, in what order, and where does that work wait?

Carried forward to Week 4

- The requirement identifiers R-01 to R-05 and Q-01, unchanged.
- Five models of B-001: the journey, the request and stored states, the data model, the R-03 table and the sequences.
- Open decisions: the fairness policy, room closures (Week 14), and how an Unknown outcome is resolved (Week 6).

In practice · examples from 2026

Teams keep diagrams as text next to the code in Mermaid or PlantUML, draw them in draw.io, and design interface states in Figma.

Sources for this lecture

S01 IEEE Computer Society, Guide to the Software Engineering Body of Knowledge (SWEBOK V4.0a) https://www.computer.org/education/bodies-of-knowledge/software-engineering

S02 ACM / IEEE-CS / AAAI, Computer Science Curricula 2023, final report https://csed.acm.org/final-report/

S08 Simon Brown, The C4 model for visualising software architecture https://c4model.com/

S09 IETF, RFC 9110: HTTP Semantics https://www.rfc-editor.org/rfc/rfc9110.html

S25 W3C, Web Content Accessibility Guidelines 2.2 https://www.w3.org/TR/WCAG22/

Terms used this week

Model
A deliberately simplified description that answers one question about a system, and says what it leaves out.

User journey
The steps a person takes toward a goal, with what they see and what is still uncertain at each step.

State and event
A state is a condition something can be in; an event is what can move it to another state.

Condition (guard)
What must hold for a transition to happen, such as R-03 allows.

Transition
A permitted change from one state to another: event, condition, outcome.

Safety invariant
A statement that stays true across every permitted transition: a forbidden state never occurs.

Progress property
A promise that pending work is eventually resolved; it needs scheduling and recovery, not only arrows.

Entity and identifier
A thing with its own identity, such as a room, found by a stable identifier like C101.

Relationship and cardinality
How entities refer to each other, and how many: a booking belongs to one room; a room has many bookings.

Referential integrity
Every reference points at something that exists: a well-formed ID is not enough.

Historical snapshot
A deliberate copy of what was true at a moment, such as the room name on a receipt.

Decision table
One row per combination of conditions, with the decision and the rule that gives it.

Sequence diagram
Participants as lifelines and numbered messages in order; distance is not time.

Interface state
What a screen promises right now: loading, empty, an error, unavailable or results, and what to do next.